Problems for First-Fit, Best-Fit, and Worst-Fit Memory Allocation Strategies

1. First-Fit Problem

Scenario:

Available memory blocks (in KB):
Block Sizes: 100, 500, 200, 300, 600

Processes to be allocated (in KB):
Process Sizes: 212, 417, 112, 426

Steps:

- **212 KB**: Fits in the first block of size **500 KB**. Remaining: **500 - 212 = 288 KB**.

  100, 288, 200, 300, 600
- **417 KB**: Fits in the first block of size **600 KB**. Remaining: **600 - 417 = 183 KB**.

  100, 288, 200, 300, 183
- **112 KB**: Fits in the first block of size **288 KB**. Remaining: **288 - 112 = 176 KB**.

  100, 176, 200, 300, 183
- **426 KB**: Cannot fit in any block.

Result:

- Processes allocated: 212 KB, 417 KB, 112 KB
- Process not allocated: 426 KB
- External fragmentation: 100 KB (first block) + 176 KB (second block) + 183 KB (last block).

2. Best-Fit Problem

Scenario:

Available memory blocks (in KB):
Block Sizes: 100, 500, 200, 300, 600

Processes to be allocated (in KB):
Process Sizes: 212, 417, 112, 426

Steps:

212 KB: Fits in the block of size **300 KB** (smallest block that fits). Remaining: **300 - 212 = 88 KB**. 100, 500, 200,88, 600

- **417 KB**: Fits in the block of size **500 KB**. Remaining: **500 - 417 = 83 KB**.

- **112 KB**: Fits in the block of size **200 KB**. Remaining: **200 - 112 = 88 KB**.
- **426 KB**: Fits in the block of size **600 KB**. Remaining: **600 - 426 = 174 KB**.

Result:

- All processes are allocated.
- External fragmentation: 100 KB (first block) + 88 KB (second block) + 83 KB (third block).

3. Worst-Fit Problem

Scenario:

Available memory blocks (in KB):
Block Sizes: 100, 500, 200, 300, 600

Processes to be allocated (in KB):
Process Sizes: 212, 417, 112, 426

Steps:

- **212 KB**: Fits in the largest block, **600 KB**. Remaining: **600 - 212 = 388 KB**.
- **417 KB**: Fits in the largest remaining block, **500 KB**. Remaining: **500 - 417 = 83 KB**.
- **112 KB**: Fits in the largest remaining block, **388 KB**. Remaining: **388 - 112 = 276 KB**.
- **426 KB**: Cannot fit in any block.

Result:

- Processes allocated: 212 KB, 417 KB, 112 KB
- Process not allocated: 426 KB
- External fragmentation: 100 KB (first block) + 83 KB (second block) + 276 KB (last block).

Key Observations:

- **First-Fit**: Simple but may leave gaps scattered in memory.
- **Best-Fit**: Minimizes leftover space but may create smaller, unusable fragments.
- **Worst-Fit**: Leaves larger fragments but may waste space when processes don't fit into smaller blocks.

	Formulate how long a paged memory reference takes if memory reference takes 200

1	nanoseconds .Assume a paging system with page table stored in memory. In a paging system where the **page table is stored in main memory** and **no TLB is used**, each memory reference requires **two memory accesses**: 1. **Accessing the page table** → 400 nanoseconds 2. **Accessing the actual memory word** → 400 nanoseconds **Total paged memory reference time** A paged memory reference takes **800 nanoseconds**.
2	Consider a **logical address space of 128 pages**, with **512 words per page**, mapped onto a **physical memory of 64 frames**. Answer the following: a. How many bits are there in the **logical address**? b. How many bits are there in the **physical address**? **Solution** **Given** • Number of pages = **128** • Page size = **512 words** • Number of frames = **64** • Frame size = same as page size = **512 words** **(a) Logical Address Size** A logical address consists of: • **Page number** • **Offset (word number within the page)** **Page number bits** • 128 pages • • Page number requires **7 bits** **Offset bits** • Page size = 512 words • • Offset requires **9 bits** **Total logical address bits** **(b) Physical Address Size** A physical address consists of: • **Frame number** • **Offset** **Frame number bits** • 64 frames • • Frame number requires **6 bits** **Offset bits** • Frame size = 512 words • Offset = **9 bits** **Total physical address bits**

	• **Logical address size = 16 bits** • **Physical address size = 15 bits**
3	In the **working set model**, consider the following **page reference string**: 3 5 2 6 8 8 8 8 6 2 5 3 4 7 2 3 4 7 7 7 4 7 4 7 7 2 4 3 4 If **Δ = 10**, determine the **working set at time t** . **Step 1: Understand the Working Set Model** The **working set** at time **t** with window size **Δ** is defined as: The **set of pages referenced during the last Δ page references up to and including time t** . Here: • Δ = 10 • We must look at the **last 10 page references** before time t . **Step 2: Identify the Last Δ = 10 References** The **last 10 references up to time t** are references **14 to 23**: 7, 2, 3, 4, 7, 7, 7, 4, 7, 4 **Step 3: Find the Working Set** The working set includes **only the unique pages** in the Δ window. From: 7, 2, 3, 4, 7, 7, 7, 4, 7, 4 The **distinct pages** are: {2, 3, 4, 7} **Working set at time t (Δ = 10):**
4	A file of size **120 KB** is stored on a disk with a **block size of 2 KB**. a) How many blocks are required to store the file? An **index block** can store **512 pointers**. If the **block size is 2 KB**, find the **maximum file size** that can be supported using indexed allocation. (a) Number of Blocks Required $$\text{Number of blocks} = \frac{\text{File size}}{\text{Block size}}$$ $$= \frac{120\ \text{KB}}{2\ \text{KB}} = \boxed{60\ \text{blocks}}$$

	(b) Maximum File Size In indexed allocation: • Each pointer refers to **one data block** • Each data block stores **2 KB** $$\text{Maximum file size} = 512 \times 2\ \text{KB}$$ $$= 1024\ \text{KB} = \boxed{1\ \text{MB}}$$ **Number of blocks required = 60 blocks** **Maximum file size = 1024 KB (1 MB)**